

Self Study

Pandas - read this on your own before the next class - about 40 minutes

Everything here is something you can learn on your own in one sitting. None of it is hard, and none of it needs me standing in front of you. Read it, type it, and come to the next class already knowing it - that way we spend our time together on the part that actually needs a teacher.

Set up first. Open a fresh notebook, in the same folder as `tips.csv` and `Titanic-Dataset.csv`:

```
import pandas as pd
```

```
df = pd.read_csv('tips.csv')          # 244 rows, 7 columns
t = pd.read_csv('Titanic-Dataset.csv') # 891 rows, 12 columns
```

Start clean - do not use the class notebook, because we added extra columns to `df` there and your outputs will not match the ones printed below.

1. Look at the table before you touch it

You have been jumping straight into `df.head()`. These three tell you far more, in less time.

`df.info()`

Every column, its type, and how many values are actually there. One command, whole picture.

```
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   total_bill  244 non-null    float64
 1   tip         244 non-null    float64
 2   sex        244 non-null    object
 3   smoker     244 non-null    object
 4   day        244 non-null    object
 5   time       244 non-null    object
 6   size       244 non-null    int64
```

The column that matters is **Non-Null Count**. Here every column says 244 out of 244 - nothing is missing. Now run `t.info()` on the Titanic and look at Age and Cabin. That is the same 177 and 687 we ended class with, visible in a single line each.

On newer versions of pandas you may see 'str' where this says 'object'. Same thing.

`df.describe()`

I ran this in class without explaining it. It gives count, mean, standard deviation, min, max and the quartiles - but **only for the number columns**. Text columns are ignored.

```
df.describe().round(2)

```

	total_bill	tip	size
count	244.00	244.00	244.00
mean	19.79	3.00	2.57
std	8.90	1.38	0.95
min	3.07	1.00	1.00
25%	13.35	2.00	2.00
50%	17.80	2.90	2.00
75%	24.13	3.56	3.00
max	50.81	10.00	6.00

Read one row of it properly: the smallest bill all week was 3.07 and the largest was 50.81, but half of all bills were under 17.80. That is the shape of the data in three numbers.

`df.dtypes` and `df.columns.tolist()`

Two one-liners that end a lot of guessing. `dtypes` tells you what type each column is; `columns.tolist()` gives you the names as a plain Python list you can read and copy.

```
df.dtypes
df.columns.tolist()
['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size']
```

`df.sample(5)` is also worth knowing - five **random** rows instead of the first five. `head()` only ever shows you the top of the file, which can be misleading if the data is sorted.

2. What is actually inside a column

You know `value_counts()`. These are its two neighbours, plus two arguments that change what it does.

`unique()` and `nunique()`

```
df['day'].unique()      # which values exist
df['day'].nunique()     # how many different ones
['Sun' 'Sat' 'Thur' 'Fri']
4
```

Use `unique()` before you write a condition. If you filter for 'Sunday' when the file says 'Sun', you get zero rows and no error - and you will stare at that for twenty minutes.

`value_counts(dropna=False)` - read this one carefully

Same function you already know. One extra argument, and the blanks stop hiding.

```
t['Embarked'].value_counts()      # the default
t['Embarked'].value_counts(dropna=False)  # count the blanks too

S      644      S      644
C      168      C      168
Q       77      Q       77
                NaN       2
```

On the left, the three numbers add up to 889. There were 891 passengers. By default `value_counts()` quietly leaves the missing ones out - exactly the same silence that broke our Titanic answers at the end of class. `dropna=False` makes them visible.

`value_counts(normalize=True)`

Percentages instead of counts.

```
(df['day'].value_counts(normalize=True) * 100).round(1)

Sat      35.7
Sun      31.1
Thur     25.4
Fri       7.8
```

3. Quick maths on a filtered column

You already used `.mean()` on a filtered column in class. Every one of these works exactly the same way, so there is nothing new to learn - only names to collect.

Function	What it gives you
<code>.sum()</code>	total
<code>.min()</code> / <code>.max()</code>	smallest / largest value
<code>.median()</code>	the middle value - often more honest than the mean
<code>.count()</code>	how many values (blanks not counted)
<code>.idxmax()</code> / <code>.idxmin()</code>	the label of the row holding the largest / smallest value

```
sat = df.loc[df['day'] == 'Sat', 'tip']
```

```
sat.sum()      # 260.4
sat.max()      # 10.0
sat.median()   # 2.75
sat.count()    # 87
```

`idxmax()` is the interesting one - it does not give you the value, it gives you **where the value is**, which you can then feed straight back into `loc`:

```
df['tip'].idxmax()      # 170 <- the row label, not the tip
df.loc[df['tip'].idxmax()]  # the whole row for that biggest tip
```

This also answers exit-quiz question 4 in one line: `df['day'].value_counts().idxmax()` gives 'Sat'.

4. Renaming columns

The Titanic file has capital letters everywhere, which is annoying to type. You can fix it once:

```
t = t.rename(columns={'Pclass': 'pclass', 'Sex': 'sex', 'Age': 'age'})
```

If you rename, rename in your very first cell and stay consistent - otherwise half your notebook says Age and half says age, and nothing runs twice.

5. The Bonus section already in your class notebook

We did not reach it in class. Read it yourself - it is fully written out, with questions B1, B2, B3 and B5 waiting for you. Everything in it is a **shorter way to do something you can already do**, which is why it is safe to learn alone.

Instead of	Write
<code>df[(df['day']=='Sat') (df['day']=='Sun')]</code>	<code>df[df['day'].isin(['Sat', 'Sun'])] -> 163 rows</code>
<code>df[(df['tip']>=3) & (df['tip']<=5)]</code>	<code>df[df['tip'].between(3,5)] -> 103 rows</code>
sorting by hand	<code>df.sort_values('tip', ascending=False)</code>
taking the top few after sorting	<code>df.nlargest(5, 'tip')</code>

And `reset_index()` - the fix for the thing that bit us in class. After filtering, the row labels stay as they were, so `sat.loc[0]` fails:

```
sat = df[df['day'] == 'Sat']
sat.index[:5].tolist()      # [19, 20, 21, 22, 23]  <- no 0 anywhere
```

```
sat = sat.reset_index(drop=True)
sat.index[:5].tolist()      # [0, 1, 2, 3, 4]
```

Skip `set_index()` at the end of that section. Leave it - we are doing it together in class, and it makes a lot more sense with a demo than on paper.

Check yourself

Six short ones. Answers on the last line - do not look until you have typed all six.

1. How many columns does `tips.csv` have, and which of them hold numbers?
2. What is the largest `total_bill`, and which row label is it on?
3. How many different values does the `time` column contain?
4. What percentage of all bills were on a Friday?
5. How many Titanic passengers have no Embarked value at all?
6. Among Saturday bills only, what is the median tip?

Answers: 1) 7 columns - `total_bill`, `tip` and `size`. 2) 50.81, row label 170. 3) 2. 4) 7.8%. 5) 2. 6) 2.75

Before the next class

Come in with three things done: this read, the class notebook re-run from top to bottom with every blank cell filled, and the assignment attempted.

Next session we pick up where we stopped - the 177 passengers with no age, and what you are supposed to do with a hole in your data.